

Criando Jogos para XBox 360 e PC utilizando XNA

Criando Jogos 2D

Bruno Pereira Evangelista

www.brunoevangelista.com

Agenda

- **Estrutura do jogo**
- **Criando Jogos 2D**
 - Sistema de coordenadas 2D
 - Sprites, Texturas
- **Desenhando sprites**
- **Entrada de dados (Controle, Teclado, Mouse)**
 - Movendo sprites com o teclado
- **Colisão entre sprites**
- **Sprites animados**

Agenda

- **Cenário scrolling**
- **Atualização baseada em tempo**
- **Escrita 2D**
- **Distribuindo jogos feitos com XNA**
- **Tópicos avançados**

Estrutura do Jogo

● Métodos da classe Game

- Initialize
- LoadGraphicsContent
- UnloadGraphicsContent
- Update
- Draw

Estrutura do Jogo

● Initialize

- Lógica de inicialização do jogo
- Criar e inicializar componentes não gráficos do jogo
- Pesquisa por serviços necessário

● Observações

- Executado antes do método LoadGraphicsContent

● Restrições

- Componentes gráficos não devem ser carregados aqui!

Estrutura do Jogo

- **LoadGraphicsContent/UnloadGraphicsContent**
 - Carrega e descarrega todos os componentes gráficos do jogo
- **Observações**
 - O XNA permite que conteúdos gerenciados e não gerenciados sejam criados!
 - Os métodos Load e Unload informam qual tipo de recurso deve ser carregado/descarregado (gerenciado ou não gerenciado)
 - Para carregar e remover de recursos gerenciados utilizamos a classe ContentManager

Estrutura do Jogo

● LoadGraphicsContent

- Carrega todos os componentes gráficos do jogo

● Observações

- O XNA permite que conteúdos gerenciados e não gerenciados sejam criados!
- Para carregar e recursos gerenciados utilizamos a classe `ContentManager`
- O parâmetro “loadAllContent” informa se todos os componentes devem ser carregados, ou apenas os não gerenciados

Estrutura do Jogo

● UnLoadGraphicsContent

- Descarrega todos os componentes gráficos do jogo

● Observações

- Nem tudo é gerenciado no XNA. Objetos alocados manualmente precisam ser removidos manualmente
- Todos os objetos alocados pelo ContentManager, podem ser removidos pelo método Unload()
- O parâmetro “unloadAllContent” informa se todos os componentes devem ser descarregados, ou apenas os não gerenciados

Estrutura do Jogo

● Update

- Lógica de atualização dos objetos do jogo

● Observações

- O método Update recebe um gameTime contendo várias informações sobre tempo
- A lógica de atualização e desenho dos objetos são executadas separadamente
- A ordem Update -> Draw sempre é respeitada, mas vários updates podem preceder um draw e vice-versa

Estrutura do Jogo

- **Draw**
 - Lógica de desenho dos objetos do jogo
- **Observações**
 - O método Draw também recebe um gameTime contendo várias informações sobre tempo
 - No final do método, a superclasse Game faz as trocas de buffers necessária para a exibir o resultado (Present)

Device Lost and Reset

● Device Lost

- Sempre que a janela é minimizada, ou perde o foco para outra janela o Device da mesma é perdido
- Todos os dados na memória de vídeo são perdidos

● Device Reset

- Quando a janela entra novamente em foco o Device precisa ser reiniciado e todos os objetos copiados novamente para a memória de vídeo
- A cópia dos recursos para a memória de vídeo é automática em recursos gerenciados

ResourceManagementMode

- **Utilizado na criação de vários objetos**
- **Automatic**
 - Recursos gerenciados
 - São copiados para a memória de vídeo conforme necessário
- **Manual**
 - Recursos não gerenciados
 - É necessário criá-los e remove-los manualmente
 - Nem tudo é gerenciado... =(

Estrutura do Jogo

- **Atributos da classe Game**
 - **GraphicsDeviceManager**
 - **ContentManager**

Estrutura do Jogo

- **GraphicsDeviceManager**
 - Utilizado para configurar e gerenciar GraphicsDevices (Dispositivos usados na renderização)
- **Exemplos de configurações**
 - Dimensões da tela
 - Formato dos buffers de cor, profundidade, etc
 - Sincronismo vertical (vsync)
 - Versão de shader mínima necessária

Estrutura do Jogo

- **ContentManager**
 - Carrega os conteúdos gerados pelo Content Pipeline
- **Observações**
 - Previne que o mesmo objeto seja carregado várias vezes, retornando sempre uma referência para o mesmo objeto

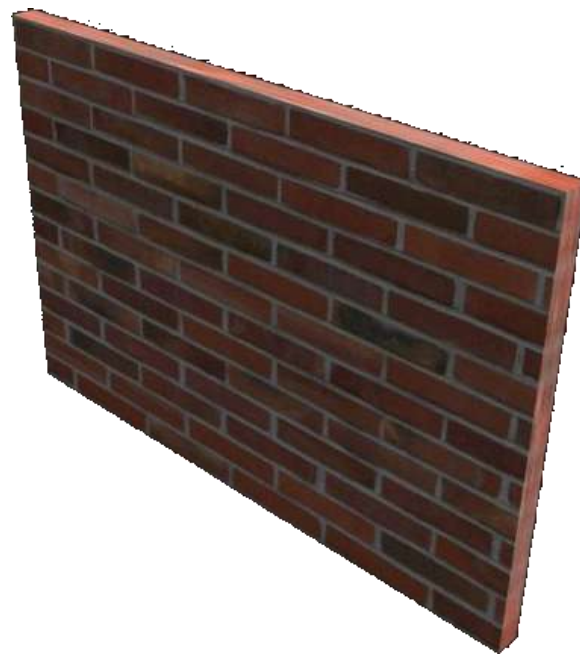
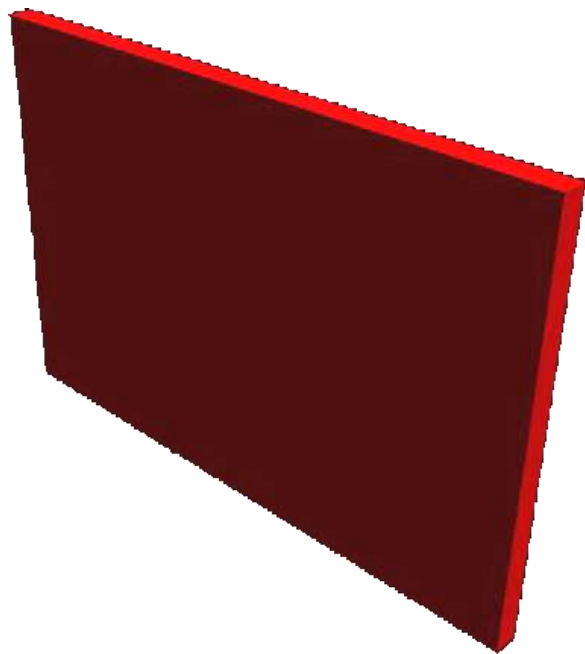
PARTE PRÁTICA

Criando Jogos em 2D

Cenários 2D

● Textura

- Uma imagem mapeada para a superfície de um objeto
- Adiciona maiores detalhes aos objetos



Cenários 2D

● Sprites

- Sprites são imagens 2D que são integradas a uma cena e podem ser manipuladas separadamente
- Utilizadas para compor cenas de jogos 2D

● Exemplo

- O cenário de fundo pode ser um sprite, e cada modelo que se move outro sprite

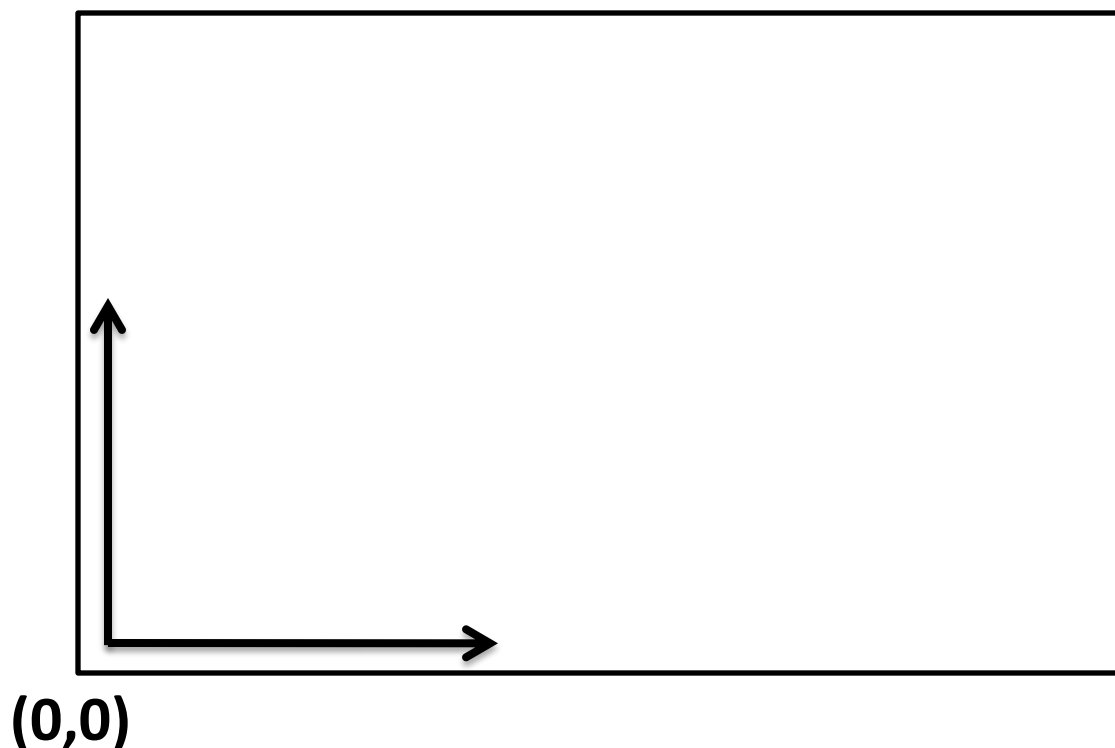


Sistema de Coordenadas 2D

- Sistema de coordenadas 2D convencional

Diagonal inferior esquerda é a origem.

(Largura, Altura)

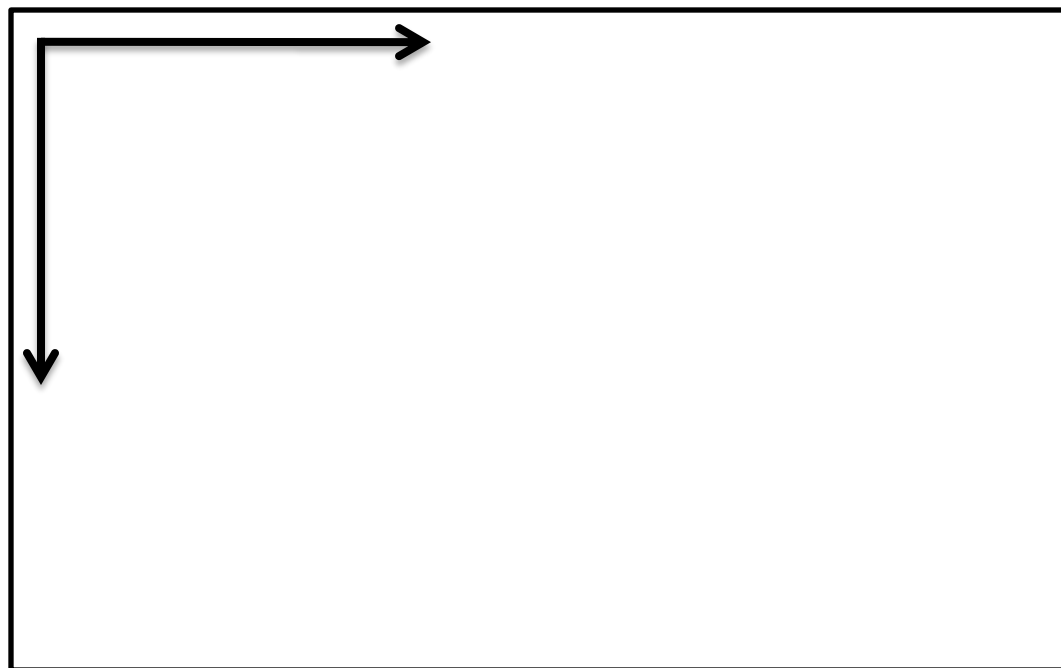


Sistema de Coordenadas 2D

- Sistema de coordenadas 2D da tela (Windows)

Diagonal superior esquerda é a origem

(0,0)



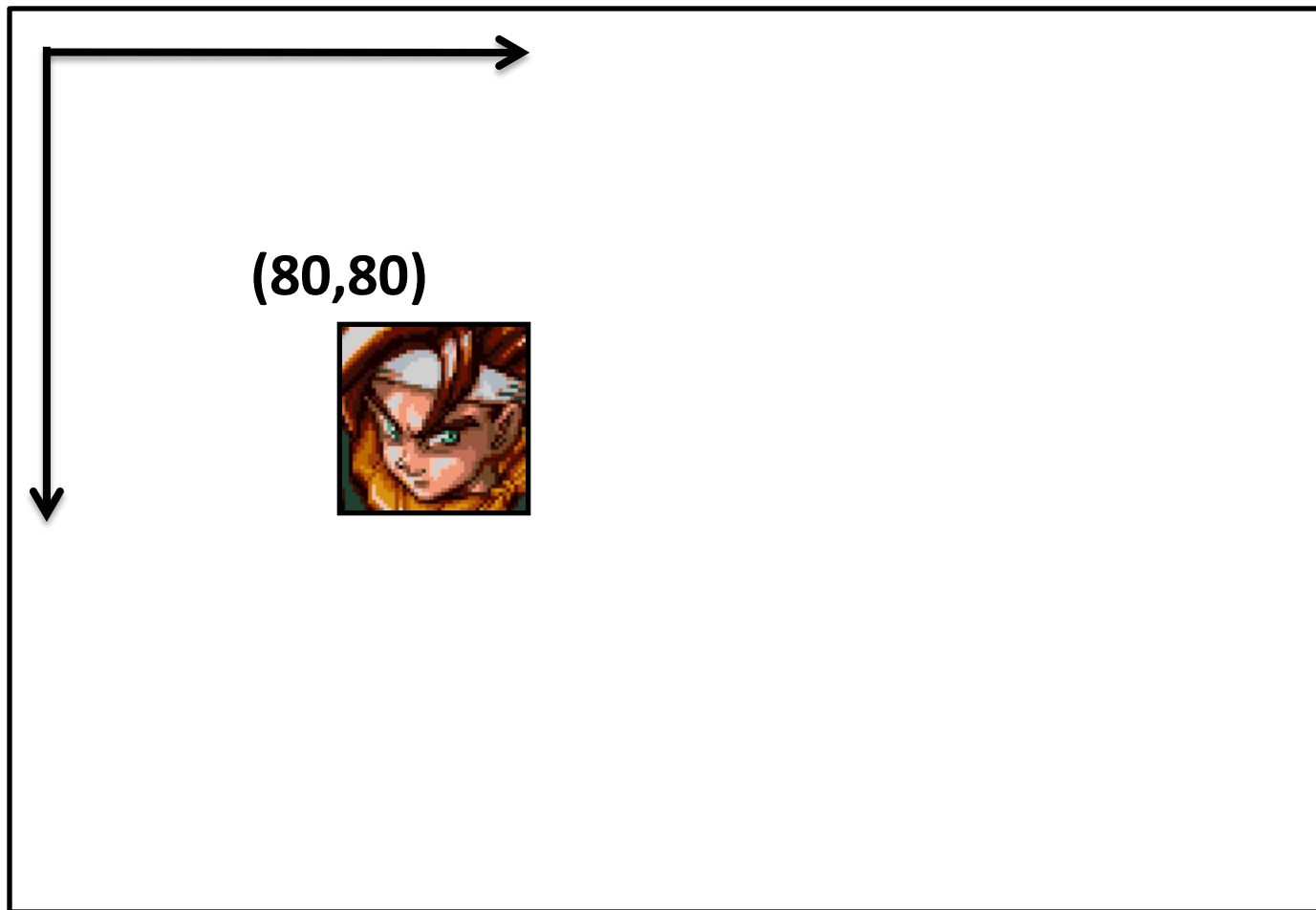
(Largura, Altura)

Sistema de Coordenadas 2D

- **O sistema de coordenada 2D da tela é o usado pelo XNA**
 - Quando for criar um Sprite é preciso passar a posição do mesmo na orientação da tela
 - A posição de desenho do sprite não é o centro do mesmo, mas sim a posição da diagonal superior esquerda do mesmo
- **Calculando a posição de desenho a partir do centro**
 - $\text{Desenho} = \text{Centro.X} - \text{Largura} * 0.5,$
 $\text{Centro.Y} + \text{Altura} * 0.5$

Sistema de Coordenadas 2D

(0,0)



(Largura, Altura)



● Carregando Texturas

- Para as texturas serem compiladas pelo Content Pipeline é preciso adiciona-las ao projeto do jogo
- As texturas podem ser carregadas através do ContentManager

● Desenhando Sprites

- Os sprites podem ser desenhados através da classe SpriteBatch

XNA - Exercício

- **Exercício 1 - Desenhando sprites na tela**
- **Passos**
 - Adicionar uma textura ao projeto
 - Carregar textura a ser aplicada no sprite
 - Criar um SpriteBatch
 - Desenhar o sprite

Exercício

- Carregando Textura

```
Texture2D texture =  
    Content.Load<Texture2D>(@“textura1”)
```

- Criando SpriteBatch

```
SpriteBatch spriteBatch = new SpriteBatch(GraphicsDevice)
```

- Desenhando sprites

```
spriteBatch.Begin()  
spriteBatch.Draw(texture, new Vector2(X, Y), Color.White)  
spriteBatch.End()
```

Exercício

- **Existem várias maneiras de desenhar um sprite**
 - **Definindo uma posição 2D onde o objeto será desenhado**
 - **Definindo um retângulo onde o objeto será desenhado**
 - **Definindo um retângulo onde o objeto será desenhado e qual parte da textura será desenhada nesse retângulo**
 - **A cor do sprite também pode ser modulada na hora do desenho**

Exercício

- Desenhando sprite

```
spriteBatch.Draw(texture, new Rectangle(X, Y,  
    texture.Width, texture.Height), Color.White)
```

- Escalando sprite

```
spriteBatch.Draw(texture, new Rectangle(X, Y,  
    texture.Width * 2, texture.Height * 2), Color.White)
```

- Modulando a cor

```
spriteBatch.Draw(texture, new Rectangle(X, Y,  
    texture.Width * 2, texture.Height * 2), Color.Red)
```

XNA - Exercício

- **Exercício 2 – Desenhar um sprite de fundo na tela e desenhar outro sprite sobre o fundo em várias posições e com tamanhos variados**
- **Passos**
 - Adicionar duas texturas ao projeto (Fundo, Personagem)
 - Carregar as texturas a serem aplicadas
 - Criar um SpriteBatch
 - Desenhar o sprite de fundo do tamanho da tela
 - Desenhar o personagem em diferentes posições e com tamanhos variados

Exercício

- Pegando viewport da tela

`Viewport viewport = graphics.GraphicsDevice.Viewport`

- Desenhando sprite de fundo

`spriteBatch.Draw(background, new Rectangle(0, 0, viewport.Width, viewport.Height), Color.White)`

- Agora é só desenhar os outros sprites!

SpriteBatch

- **É possível configurar como o SpriteBatch é executado quando o mesmo é iniciado**
 - **SpriteBlendMode:** Configura como o desenho do sprite é misturado com a imagem já gravada na tela
 - **SpriteSortMode:** Configura a ordem em que os sprites são desenhados, ou se o desenho é imediato
 - **SaveStateMode:** Permite salvar e restaurar o estado do **GraphicsDevice**
 - **Matrix:** Matrix de transformação a ser aplicada ao Sprite

Mudando resolução da tela

- O tamanho da tela é uma configuração do GraphicsDevice
 - Pode ser alterada pela classe GraphicsDeviceManager
 - Caso o GraphicsDevice já tenha sido criado é necessário chamar o método ApplyChanges() após alterar as configurações
- Exemplo:
`graphics.PreferredBackBufferWidth = 640`
`graphics.PreferredBackBufferHeight = 480`
`graphics.ApplyChanges()`

Entrada de Dados

- **Utilizadas para fornecer informações do usuário ao jogo**
- **Tipos de entradas suportados pelo XNA**
 - **Controle Xbox 360 (Windows e Xbox 360)**
 - **Teclado (Windows e Xbox 360)**
 - **Mouse (Windows)**

Entrada de Dados

- **Os dados de entrada dos dispositivos podem ser obtidos apartir do estado dos mesmos**
 - O estado do dispositivo é a imagem do estado de todas as “entradas” do dispositivo em um determinado momento
 - Por exemplo, o estado do teclado é o estado de todas as suas teclas em um determinado momento
- **Problemas**
 - Não existem eventos que indiquem quando a tecla foi pressionada ou solta
 - Para verificar se uma tecla foi pressionada uma vez é preciso armazenar o estado anterior do teclado

Entrada de Dados

- **Classes de estado dos dispositivos**
 - **GamePadState**
 - **KeyboardState**
 - **MouseState**
- **Classes dos dispositivos (Utilizadas para ler o estado dos mesmos)**
 - **GamePad**
 - **Keyboard**
 - **Mouse**

XNA - Exercício

- **Exercício 3 – Mover o sprite desenhado com o teclado**
- **Passos**
 - Criar uma variável para armazenar a posição do sprite na tela (Exemplo: `int spritePositionX, spritePositionY`)
 - Ler o estado do teclado
 - Alterar a posição do sprite de acordo com as teclas pressionadas

Exercício

- **Lendo o estado do teclado**

```
KeyboardState keyboardState = Keyboard.GetState()
```

- **Processando teclas pressionadas**

```
if (keyboardState.IsKeyDown(Keys.Right))
```

```
    spritePositionX += 1
```

```
.... ....
```

- **Desenhando o sprite**

```
Rectangle rect = new Rectangle(  
    spritePositionX, spritePositionY,  
    spriteTexture.Width, spriteTexture.Height) )
```

```
spriteBatch.Draw(spriteTexture, rect, Color.White)
```

Colisão entre sprites

- A colisão entre sprites é usada para definir as áreas para onde o sprite pode se mover
- A colisão entre sprites pode ser feita de várias maneiras
 - Colisão entre o retângulos dos sprites
 - Colisão entre os pixels dos sprites
- Nesta parte utilizaremos a primeira abordagem que é mais simples

Colisão entre sprites

- Colisão entre retângulos
 - Mais fácil e rápida de ser calculada
 - Não garante que as texturas dentro do sprite colidem



Não há colisão



Colisão

Colisão entre sprites

Colisão entre retângulos

- Cada retângulo possui um X inicial (rectStartX) e um X final (rectEndX), e também um Y inicial e um Y final

Como verificar a colisão?

- No eixo X

`(rect1EndX >= rect2StartX && rect1StartX <= rect2EndX)`

- No eixo Y

`(rect1EndY >= rect2StartY && rect1StartY <= rect2EndY)`

- Para haver colisão ambos os testes devem retornar verdadeiro

XNA - Exercício

- **Exercício 4 – Fazer detecção de colisão entre os sprites**
- **Passos**
 - Criar um novo sprite e definir sua posição na tela
 - Criar um método para detectar colisão entre dois retângulos
 - Na atualização do jogo: Salvar a posição do sprite, atualizar a posição de acordo com a entrada. Quando houver colisão, restaurar a posição para a posição salva

Exercício

- **Atualizando a posição do jogador**

`Vector2 savedPosition = spritePosition`

`if (keyboardState.IsKeyDown(Keys.Right))`

`spritePosition.X += 0.5f * elapsedTimeMillis`

`... ..`

`Rectangle rect1 = new Rectangle(spritePositionX,
spritePositionY, spriteTexture.Width, spriteTexture.Height)`

`Rectangle rect2 = new Rectangle(enemyPositionX,
enemyPositionY, enemyTexture.Width, enemyTexture.Height)`

`if (IsCollision(spriteRect, enemyRect))`

`spritePosition = savedPosition;`

Animação

- Para criar o efeito de animação é preciso trocar a imagem exibida rapidamente
- Para a animação de personagens 2D precisamos utilizar várias imagens do personagem
 - Cada imagem é um quadro da animação
 - A troca das imagens dá o efeito de animação do jogador



Animação

- Para realizarmos a animação utilizando XNA precisamos carregar várias texturas do personagem
- A textura utilizada para desenho no SpriteBatch deve ser trocada de acordo com algum intervalo de tempo

XNA - Exercício

- **Exercício 4 – Desenhar sprites animados**
- **Passos**
 - Carregar um arranjo de texturas
 - Criar variáveis para controlar o tempo decorrido e o número da textura a ser exibida
 - Desenhar o sprite

Exercício

- Carregar um arranjo de texturas

```
Texture2D[] spriteTexture = new Texture2D[4];  
for (int i = 0; i < spriteTexture.Length; i++)  
    spriteTexture[i] = content.Load<Texture2D>(  
        @"anim1\run" + (i+1) ) )
```

Exercício

- **Atualizando textura a ser exibida**

`elapsedTime += time.ElapsedGameTime.Milliseconds`

```
if (elapsedTime > CHANGE_ANIMATION_TIME) {  
    currentFrame = (currentFrame + 1) % spriteTexture.Length  
    elapsedTime = 0  
}
```

- **Desenhando**

`spriteBatch.Draw(spriteTexture[currentFrame], ....)`

Cenário Scrolling

- **Muitos jogos 2D utilizam cenários com scrolling**
 - O personagem permanece parado no centro da tela enquanto o cenário se move
- **Exemplo**
 - Vários jogos de RPG 2D
 - Jogos de nave, onde a nave se move e o cenário também

XNA - Exercício

● Exercício 5 – Criar um cenário com scrolling

O personagem fica parado no centro da tela e somente o cenário se move

● Passos

- Posicionar o sprite do player no centro da tela
- Criar uma variável para armazenar o deslocamento da imagem de fundo (background)
- Alterar a posição do fundo de acordo com as teclas pressionadas
- Utilizar o método Draw com source e destination, para mudar a parte da textura que é exibida
- Mudar a configuração do mapeamento de textura

Exercício

- **Posicionar o sprite player no centro da tela**

```
spritePositionX = (int)(  
    (viewport.Width - spriteTexture.Width) * 0.5f )
```

```
spritePositionY = (int)(  
    (viewport.Height - spriteTexture.Height) * 0.5f)
```

- **Processando teclas pressionadas**

```
if (keyboardState.IsKeyDown(Keys.Right))
```

```
    backgroundPositionX += 1
```

```
.... .... (Agora devemos mover APENAS o background)
```

Exercício

- **Desenhando o fundo em separado**

```
spriteBatch.Begin(SpriteBlendMode.AlphaBlend,  
    SpriteSortMode.Immediate,  
    SaveStateMode.SaveState)
```

```
spriteBatch.Draw(backgroundTexture, new Rectangle(0, 0,  
    viewport.Width, viewport.Height), new  
    Rectangle(backgroundPositionX, backgroundPositionY,  
    backgroundTexture.Width, backgroundTexture.Height),  
    Color.White)
```

```
spriteBatch.End()
```

Cenário Scrolling

Background 1



Background 2



Background 3



Background 4



Janela

Exercício

- **Mudando mapeamento de textura**
 - O mapeamento de textura padrão utilizado no SpriteBatch é CLAMP (Repete o último pixel)
 - Para fazermos scrolling é necessário repetir a textura, o mapeamento de textura que faz isso é o Wrap
- **Código**
 - Após iniciar o SpriteBatch em modo imediato é possível alterar as configurações

```
GraphicsDevice.SamplerStates[0].AddressU =  
    TextureAddressMode.Wrap
```

```
GraphicsDevice.SamplerStates[0].AddressV =  
    TextureAddressMode.Wrap
```

Cenário Scrolling

- **Modos de criar um cenário scrolling**
 - Modificando o mapeamento de texturas é fácil criar um mapa com scrolling em todas as direções
 - Outra maneira possível seria desenhar o fundo mais de uma vez, em diferentes posições
- **Exemplo**
 - Em um jogo de naves o fundo precisaria ser desenhado 2 vezes, pois a tela deslocaria somente no eixo Y
 - Em um jogo onde você pode mover em qualquer direção seria necessário desenhar o fundo 4 vezes

Atualização baseada em tempo

- O método Update é utilizado para modificar a posição dos objetos

- Exemplo: (Movimento em X)

- $\text{Posição.X} = \text{Posição.X} + 1$

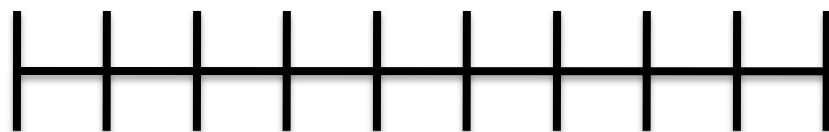
- Problema

- O número de vezes que o método Update é chamado em um segundo não é fixo
 - 10 fps: Objeto na posição 10
 - 60 fps: Objeto na posição 60

Atualização baseada em tempo

- Como tornar a distância percorrida independente da velocidade que o jogo está sendo executado?
 - Em qualquer computador após 1 segundo, o objeto deve ter movido a mesma distância
- Solução
 - Atualizar a posição do objeto baseado no tempo decorrido, entre os métodos Update

Atualização baseada em tempo

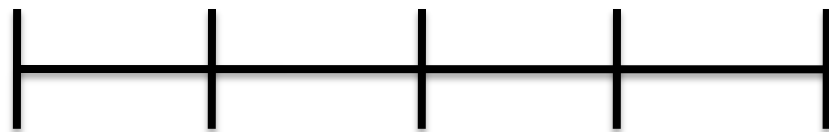


Update
10x por segundo
 $1s / 10 = 100ms$

100ms decorrido entre cada Update

$Posição.X = Posição.X + 100$ (Tempo Decorrido)

Atualizada 10x (Total = 1000)



Update
5x por segundo
 $1s / 5 = 200ms$

200ms decorrido entre cada Update

$Posição.X = Posição.X + 200$ (Tempo Decorrido)

Atualizada 5x (Total = 1000)

XNA - Exercício

- **Exercício 6 – Atualizações baseadas em tempo**
- **Passos**
 - Trocar a posição do background armazenada como int para um Vector2
 - Na atualização utilizar o tempo que decorreu desde a ultima atualização para mover o sprite
 - Na hora de desenhar truncar o valor para inteiro

Exercício

- **Atualização baseada em tempo**

```
int elapsedTimeMillis =  
    gameTime.ElapsedGameTime.Milliseconds
```

```
if (keyboardState.IsKeyDown(Keys.Right))  
    backgroundPosition.X += 0.5f * elapsedTimeMillis
```

Escrevendo na tela

- O XNA possui várias classes que facilitam a escrita na tela utilizando qualquer tipo de fonte
- As fontes são criadas através de do componente **SpriteFonts**
 - São arquivos XML que descrevem como construir uma textura utilizando uma determinada font de texto
 - Podem ser criados pela IDE do GameStudioExpress, ou adicionados manualmente
- A classe **SpriteBatch** já possui métodos para desenhar textos utilizando um **SpriteFont**

Escrevendo na tela

- **SpriteFonts possuem várias configurações**
 - **FontName: Nome da font**
 - **Size: Tamanho da font**
 - **Spacing: Espaçamento da font**
 - **Style: Estilo (Bold, Italic, etc)**
 - **CharacterRegions: Caracteres suportados pela font**

XNA - Exercício

- **Exercício 6 – Escrevendo na tela**
- **Passos**
 - **Criar um SpriteFont**
 - **Carregar o SpriteFont utilizando o ContentManager**
 - **Utilizar o SpriteBatch para escrever na tela**

Exercício

- **Carregando SpriteFont**

`spriteFont = content.Load<SpriteFont>(@"font")`

- **Escrevendo na tela**

`spriteBatch.DrawString(spriteFont, "Texto teste!",
Vector2.Zero, Color.White)`

Distribuindo Jogos

- **Jogos em XNA podem ser distribuídos para PC e Xbox**
- **Distribuição no Xbox**
 - Restrita a assinantes da Live e do Creator's Club
 - Todos os arquivos podem ser compactados em um único arquivo que pode ser distribuído
 - No entanto, para rodar o jogo no Xbox é necessário fazer um deploy do mesmo através do Game Studio Express =(
- **Aguardem futuras versões do XNA**

Distribuindo Jogos

● Distribuição no PC

- Não existe restrição, pode ser distribuído para qualquer usuário do Windows XP (SP2) ou Vista
- Possui uma grande dependência com bibliotecas da Microsoft (.NET Framework 2.0, DirectX 9.0c, XNA)
- Necessita de placas gráficas com suporte a shaders (shader model 2.0 recomendado)

Distribuindo Jogos

● **Installer for XNA Games**

- **Script para o instalador NSIS (gratuito), utilizado para criar instalações para jogos em XNA**
- **Verifica as dependências com o .NET Framework, DirectX e XNA antes de instalar o jogo**
- **Código fonte aberto, pode ser usado livremente**

● **Link:**

- **<http://www.brunoevangelista.com/projects.html>**

Outros Tópicos

- **Tile Mapping usando XNA**

Perguntas?

Bruno P. Evangelista

bpevangelista@gmail.com

Home Page

www.brunoevangelista.com

**"De fato, que aproveitará ao homem ganhar o mundo inteiro mas perder sua alma?",
Matheus 16, 26**